

SYSTEM DOCUMENTATION — CONFIDENTIAL

Sycamore Agricultural Services Cooperative

Complete System Documentation — Admin Dashboard, Member Portal, Public Website & Laravel API

MANAGED BY

Stalwart Services Limited

DOCUMENT DATE

June 2026

SYSTEM VERSION

1.1 — Member Portal Release

STATUS

Active Development

TABLE OF CONTENTS

- | | |
|----------------------------|---------------------------|
| 1. System Overview | 9. Public Website Guide |
| 2. Folder Structure | 10. API Reference Summary |
| 3. Tech Stack | 11. Database Overview |
| 4. Local Development Setup | 12. Deployment Guide |
| 5. Test Credentials | 13. What Is Complete |
| 6. How the System Works | 14. What Is Remaining |
| 7. Admin Dashboard Guide | 15. Known Issues & Notes |
| 8. Member Portal Guide | |

1 System Overview

Sycamore is a digital management platform for **Sycamore Agricultural Services Cooperative** — a real SACCO (Savings and Credit Cooperative Organization) based in Zambia, managed by **Stalwart Services Limited**. This is a **single-tenant system** — built exclusively for Sycamore cooperative.

Part	Purpose	Local URL
Laravel API	Backend — all business logic, database, authentication	<code>http://localhost:8000/api</code>
Admin Dashboard	Staff/admin interface — manage members, loans, savings, reports	<code>http://localhost:5173</code>
Member Portal	Self-service section for members — view accounts, apply for loans, request withdrawals	<code>http://localhost:5173/my</code>
Public Website	Public-facing — information, news, membership applications	<code>http://localhost:5174</code>

How a SACCO Works

- Members save a fixed amount every month (contributions)
- All member savings form a shared pool
- Members borrow from this pool at low interest rates (limited to 3× their savings balance)
- Interest earned stays in the cooperative
- At year end, profits are distributed as dividends proportional to each member's savings
- The cooperative is member-owned — one member, one vote

2

3 Tech Stack

Layer	Technology	Notes
Backend language	PHP 8.3	<code>C:\laragon\bin\php\php-8.3.26-Win32-vs16-x64\php.exe</code>
Backend framework	Laravel 11	Sanctum token auth, Eloquent ORM
Database (local)	MySQL / MariaDB	via Laragon — database name: <code>lms</code>
Database (production)	PostgreSQL	via Railway
UI Framework	React 19	Both admin dashboard and public website
Build tool	Vite 8	
CSS	Tailwind CSS 3	Emerald/green palette for admin; bush green for website
Routing	React Router v7	
State management	Redux Toolkit	Auth state (user, token)
HTTP Client	Axios	
Icons	Lucide React	
Charts	Recharts	Admin dashboard only
Node.js	v22	<code>C:\laragon\bin\nodejs\node-v22\node.exe</code>

4 Local Development Setup

Prerequisites

- Laragon running (provides PHP 8.3 + MySQL)
- Node.js v22 installed
- `lms` database created in MySQL (via phpMyAdmin or HeidiSQL)

Quick Start — All Services

Double-click `c:\laragon\www\lms\start.bat` — opens three terminal windows:

Service	URL	Command
Laravel API	<code>http://localhost:8000</code>	<code>php artisan serve</code> in <code>lms/lms/</code>
Admin Dashboard + Member Portal	<code>http://localhost:5173</code>	<code>npm run dev</code> in <code>lms/frontend/</code>
Public Website	<code>http://localhost:5174</code>	<code>npm run dev</code> in <code>lms/website/</code>

First-Time Setup Commands

```
# 1. Install Laravel PHP dependencies
cd c:\laragon\www\lms\lms
composer install

# 2. Run all database migrations (27 migrations)
php artisan migrate

# 3. Create the storage symlink (for profile photos and uploads)
php artisan storage:link

# 4. Install admin dashboard dependencies
cd c:\laragon\www\lms\frontend
npm install

# 5. Install website dependencies
cd c:\laragon\www\lms\website
npm install
```

Key Environment Variables

Backend — file: `lms/lms/.env`

```
APP_URL=http://localhost:8000
DB_DATABASE=lms
DB_USERNAME=root
DB_PASSWORD=          ← empty for Laragon default
FILESYSTEM_DISK=public
```

Admin Frontend — file: `lms/frontend/.env`

```
VITE_API_URL=http://localhost:8000/api
```

Public Website — file: `lms/website/.env`

```
VITE_API_URL=http://localhost:8000/api
```

Storage Symlink Required

Run `php artisan storage:link` once after setup. This creates the `public/storage` symlink so uploaded profile photos and documents are served correctly.

5 Test Credentials

These accounts are for local development and testing only.

Admin / Staff Dashboard — `http://localhost:5173`

ADMIN ACCOUNT

Email : omrihabeeenzul@gmail.com
Password : (set during initial setup / database seed)
Role : admin — full access to all features

Member Portal — `http://localhost:5173/login` → redirects to `/my`

TEST MEMBER ACCOUNT

Email : member@sycamore.test
Password : Member123
Linked to: Borrower ID 1 (James H)

Creating New Member Accounts

Members are auto-created when a Borrower is approved. Go to
Members

→ open a pending member → click

Approve

. A modal shows the generated login email and temporary password (format: `ABCD123`). If the member's email is already in use by a staff account, the login email falls back to

`[phone]@sycamore.member` .

6 How the System Works

User Roles & Permissions

Role	Access Level
<code>admin</code>	Full access to everything. Can manage staff permissions, loan products, settings. The only role that can see the Settings page and Loan Products page.
<code>manager</code> / <code>loan_officer</code> / <code>cashier</code> / <code>staff</code>	Access restricted by permission flags granted by admin. Cannot see Settings or Loan Products.
<code>member</code>	Member portal only (<code>/my/*</code>). Can view own accounts, apply for loans, request withdrawals.

Staff Permissions

Admin goes to **Staff & Access** to create staff accounts and set what each person can see. Each permission is a toggle:

- Members, Contributions, Share Capital, Groups
- Savings, Loans, Repayments, Dividends
- Expenses, Welfare Fund, Reports, Governance
- Inbox, News

Staff cannot see **Settings** (prevents self-promotion to admin) or **Loan Products** (interest rates are admin-only).

Authentication Flow

1. User logs in with email + password at `/login`
2. Laravel Sanctum issues a secure API token
3. Token stored in `localStorage` ; sent with every request as `Authorization: Bearer {token}`
4. After login: members are redirected to `/my` , staff/admin to `/`

Member Lifecycle

```
Website Form Submitted (status = pending)
      ↓
Admin Reviews Application in Dashboard
```

```

      ↓
Orientation Conducted, Agreement Signed
      ↓
Admin Clicks "Approve" → status = active
Member Portal Account Auto-Created (email + generated password shown)
      ↓
Member Makes First Contribution + Savings Deposit
      ↓
Monthly Contributions Continue
      ↓
Member Eligible for Loans (up to 3× savings balance)
      ↓
Dividends Paid at Year End

```

Loan Eligibility — 3× Savings Rule

A member cannot borrow more than **3 times their total savings balance**. This applies both when staff create a loan and when a member applies from the member portal. Example: if a member has K2,000 in savings, they can borrow a maximum of K6,000.

Loan Lifecycle

```

Applied (pending) → Approved → Disbursed → Active → [Repayments] → Closed
                  ↘ Rejected                               ↘ Overdue / Written Off

```

- **Interest methods:** Flat Rate or Reducing Balance
- **Repayment frequencies:** Daily, Weekly, Biweekly, Monthly, Quarterly
- **Schedule generation:** Full amortization table created automatically on disbursement

Savings & Withdrawals

- Each member has one or more Savings Accounts (separate from contributions)
- Supports deposits, withdrawals, and annual interest posting
- **Withdrawal workflow:** member or staff submits a request → admin/staff approves → savings balance is deducted and a transaction is recorded. Members cannot directly withdraw; all withdrawals require approval.

Welfare Fund

- Members pay a welfare contribution per period (e.g., monthly)
- Members or next-of-kin can submit claims (bereavement, medical, emergency, other)
- Admin reviews and approves/rejects claims, sets the approved amount
- Admin marks approved claims as paid when the money is disbursed

Savings Compliance

The Savings Compliance page shows which members are meeting their agreed monthly savings commitment. Staff can select any month and see a progress bar per member showing contributions received vs. their commitment amount.

Contributions vs Savings Accounts

- **Contributions** — monthly SACCO savings recorded by staff (tracked against a commitment amount)
- **Savings Accounts** — a flexible savings account per member with full transaction history

Dividends

At year end, admin calculates dividends based on total cooperative profit and each member's proportional savings. The system computes per-member amounts, allows review, approval, and marks as distributed once paid.

Login URL: `http://localhost:5173` | Admin account required for full access.

Dashboard

Overview cards: total members, active loans, total savings, outstanding balance, loan collection rate, overdue count. 6-month disbursement trend chart. Recent loans and latest repayments.

Members

- List with search (name/phone), status filter (pending / active / inactive / suspended / rejected)
- Member detail: full profile, tabs for Loans, Savings, Shares, Contributions, Documents
- Approve / Reject pending applications — approval auto-creates a member portal account
- Printable member statement
- Documents: upload NRC copy, pay slips, signed agreements

Savings Compliance

Select a month → see all active members with progress bars showing contributions received vs. their monthly commitment. Highlights members who are behind.

Loan Products (*Admin only*)

Define templates: name, interest rate, interest method (flat/reducing), min/max amounts, min/max term, processing fees. Staff cannot see or edit these.

Loans

Action	Description
Create Loan	Select member and product; preview full repayment schedule before submitting
Approve / Reject	Required before disbursement; checks share value eligibility
Disburse	Confirm payment made; full amortization schedule generated
Record Repayment	Enter amount received; printable receipt available
Top-Up	Add principal to an active loan
Restructure	Modify terms (term, rate, frequency) of an active loan
Write-Off	Mark irrecoverable loans
Guarantors / Collateral / Charges	All managed from the loan detail page

Savings

View all savings accounts, record deposits/withdrawals, post annual interest. Each account shows full transaction history.

Withdrawal Requests

Pending withdrawal requests from members (submitted via member portal) or staff. Approve → savings balance deducted + transaction recorded. Reject → no balance change. Filter by status (pending / approved / rejected).

Contributions

Record monthly contributions per member. View per-member contribution history. Manage contribution schedules (committed monthly amount per member).

Welfare Fund

- **Summary:** total contributions collected, total disbursed, current fund balance, pending claims count
- **Contributions tab:** record member welfare contributions per period (YYYY-MM)
- **Claims tab:** review submitted claims, approve/reject with notes and amount, mark approved claims as paid

Reports

Report	What It Shows
Collections Sheet	All repayments in a date range
Portfolio Report	Active loans — totals and outstanding balances
Aging Analysis	Overdue loans grouped by days overdue
Officer Performance	Repayments grouped by loan officer
Income Statement	Revenue (interest, fees) vs. expenses
Balance Sheet	Assets, liabilities, and member equity

Governance

- **Board members** — record elected officials with their roles and terms
- **Meeting minutes** — store minutes from board and general meetings

Inbox

View contact form submissions from the public website. Reply directly, and see live chat sessions from the website chat widget.

News

Create, edit, and publish news/announcements. Published articles are displayed on the public website's News page.

Staff & Access *(Admin only)*

Create staff accounts and manage their permissions. Each staff member gets a generated temporary password. Admin controls exactly which sections each staff member can access via 14 permission toggles.

Settings *(Admin only)*

- **Company:** SACCO name, contact information, logo
- **Users:** legacy user management (use Staff & Access for new staff)
- **SMS logs:** history of all system SMS messages sent

Profile (All Users)

Every logged-in user (staff and members) can access [/profile](#) to: edit name and phone, upload a profile photo, and change their password. The profile photo appears in the sidebar. Email address cannot be changed.

8 Member Portal Guide

URL: `http://localhost:5173/login` → after login, redirected to `http://localhost:5173/my`

The member portal is part of the same React app as the admin dashboard but uses a separate layout and separate routes (`/my/*`). Members only see their own data.

Member Portal Pages

Page	URL	What It Shows
Overview	<code>/my</code>	Savings balance, max loan allowed (3× savings), active loans count, next repayment due, overdue alert
My Loans	<code>/my/loans</code>	All loans with status; expandable repayment schedule per loan; Apply for Loan button
My Savings	<code>/my/savings</code>	All savings accounts with transaction history; Withdraw button per account; withdrawal request history
Contributions	<code>/my/contributions</code>	All monthly contributions on record
Shares	<code>/my/shares</code>	Share capital holding
Profile	<code>/profile</code>	Edit name/phone, upload photo, change password

Applying for a Loan (Member)

1. Go to **My Loans** → click **Apply for Loan**
2. Select a loan product — terms are pre-filled from the product
3. Enter amount — system shows the 3× savings maximum
4. Enter repayment term (within the product's min/max)
5. Optionally add a purpose/note
6. Submit — loan created with **pending** status, goes to admin/staff for approval

Requesting a Withdrawal (Member)

1. Go to **My Savings**
2. Click **Withdraw** on the savings account
3. Enter amount (cannot exceed current balance) and reason

4. Submit — creates a pending withdrawal request for admin/staff to approve
5. Status tracked in the Withdrawal Requests table below the savings accounts

Note

Funds are not released immediately. All withdrawals require approval by office staff. Only one pending withdrawal request is allowed per savings account at a time.

9 Public Website Guide

Local dev URL: `http://localhost:5174`

Pages

Page	Path	Key Content
Home	/	Hero, statistics, product overview, 5-step join process, cooperative values, CTA
About	/about	Who we are, mission & vision, milestones, governance structure
Products & Services	/products	Savings accounts, loan types, farm investment pool, benevolent fund
Membership	/membership	Member benefits, 5-stage onboarding process, application form
News	/news	Fetches from <code>GET /api/public/news</code> — published news from the admin dashboard
Contact	/contact	Contact information and contact form → inbox in admin dashboard

Membership Application Flow

1. Visitor fills the form on `/membership`
2. Sent to `POST /api/public/membership-applications` (no login needed)
3. A **pending** Borrower record is created in the database
4. Staff see it in **Members** (filter: pending)
5. Staff follow up, conduct orientation, activate the member → portal account auto-created

Design System

Element	Value
Primary colour	Bush green — dark: #193c27 / medium: #265c3c / mid: #2d7249
Accent colour	Earthy amber — #d4841f (CTAs and highlights)
Heading font	Montserrat (Bold/ExtraBold)
Body font	Inter (Regular/Medium)
Logo file	website/public/logo.svg

Base URL: `http://localhost:8000/api`

Public (No Authentication)

Method	Endpoint	Description
GET	/health	Health check for deployment platforms
POST	/auth/login	Login — returns bearer token + user object
POST	/public/membership-applications	Submit membership application from website
POST	/public/contact	Submit contact form message
GET	/public/news	Fetch published news articles
POST	/public/chat	Send live chat message
GET	/public/chat/{sessionId}	Poll for new chat messages

Protected (Bearer Token Required)

Area	Endpoints
Auth	POST /auth/logout GET /auth/me
Profile	GET/PUT /profile POST /profile/photo POST /profile/change-password
Dashboard	GET /dashboard
Members	Full CRUD /borrowers PATCH /borrowers/{id} (includes approval)
Loan Products <i>(admin)</i>	Full CRUD /loan-products
Loans	Full CRUD + /approve, /reject, /disburse, /top-up, /restructure, /write-off, /preview-schedule
Repayments	GET /repayments GET/POST /loans/{id}/repayments
Savings	GET/POST /savings /deposit, /withdraw, /post-interest per account
Withdrawal Requests	GET/POST /withdrawal-requests POST /{id}/approve POST /{id}/reject
Shares	GET/POST /members/{id}/shares GET /shares/summary
Contributions	GET/POST /contributions per-member view schedules
Welfare Fund	GET /welfare/summary contributions CRUD claims CRUD + review + mark-paid
Expenses	Full CRUD /expenses
Groups	Full CRUD /groups + member add/remove
Guarantors / Collateral / Charges	Per-loan management
Documents	Upload/download/delete per borrower and per loan
Dividends	Calculate, store, approve, distribute
Governance	Board members + meeting minutes
Reports	6 reports under /reports/*
News	Full CRUD /news (admin/staff)
Inbox	Contact messages + live chat threads
Staff <i>(admin only)</i>	GET/POST /staff PUT/DELETE /staff/{id}

Settings (<i>admin only</i>)	Company, users, SMS logs under /settings/*
Member Portal	GET /my/dashboard, /my/loans, /my/savings, /my/contributions, /my/shares, /my/withdrawal-requests POST /my/loans (apply)

Database Overview

Local database: `lms` (MySQL) | Production: PostgreSQL via Railway | 27 migrations applied

Table	Purpose
companies	SACCO tenants — currently one row (Sycamore, company_id = 1)
users	Staff and member accounts. Key columns: <code>role</code> (admin/staff/manager/loan_officer/cashier/member), <code>permissions</code> (JSON), <code>borrower_id</code> (links member to borrower record), <code>photo</code> (avatar path)
borrowers	Members — personal details, employment, status, monthly_savings_commitment
next_of_kin	Emergency contacts per member
loan_products	Loan templates — interest rate, method, fees, term limits
loans	Individual loan records — full lifecycle tracked here
loan_schedules	Auto-generated repayment schedule per loan
repayments	Actual payments received from members
loan_charges	Fees and penalties on loans
guarantors	Members guaranteeing other members' loans
collateral	Assets pledged against loans
savings_accounts	Savings account per member
savings_transactions	Deposits, withdrawals, interest postings on savings accounts
withdrawal_requests	Member withdrawal requests — pending / approved / rejected workflow
member_shares	Share capital per member
share_transactions	Share purchase history
contributions	Monthly contribution records per member
contribution_schedules	Expected monthly contribution commitment per member
welfare_contributions	Member welfare fund contributions per period (YYYY-MM)
welfare_claims	Welfare claims — type (bereavement/medical/emergency/other), review, paid status
expenses	Cooperative operating expenses
groups	Member lending or savings groups

group_members	Pivot — which members belong to which group
dividends	Year-end dividend declarations
dividend_allocations	Per-member dividend amounts per declaration
board_members	Elected officials (governance)
meeting_minutes	Board and general meeting records
news	News/announcement articles published to the public website
contact_messages	Contact form submissions from the website
chat_sessions / chat_messages	Live chat sessions from the website chat widget
documents	Uploaded files — polymorphic (attached to borrowers or loans)
sms_logs	Log of all SMS messages sent by the system
activity_logs	Audit trail of system actions
personal_access_tokens	Sanctum API authentication tokens

Current State

Part	Status	URL
Public Website	Live Preview	Netlify URL (shared with team)
Admin Dashboard + Member Portal	Local Only	http://localhost:5173
Laravel API	Local Only	http://localhost:8000

Full Production Deployment Steps

STEP 1 — PUSH CODE TO GITHUB

```
cd c:\laragon\www\lms
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/your-org/sycamore.git
git push -u origin main
```

STEP 2 — DEPLOY API TO RAILWAY

1. Create account at [railway.app](#)
2. New Project → Deploy from GitHub → select repo, root directory: `lms/lms`
3. Add a PostgreSQL plugin in the Railway dashboard
4. Set required environment variables (see below)
5. Railway builds using the Dockerfile and deploys automatically

```
APP_ENV=production
APP_KEY=          ← generate with: php artisan key:generate --show
APP_DEBUG=false
DB_CONNECTION=pgsql
FILESYSTEM_DISK=public
# DB_HOST / DB_DATABASE / DB_USERNAME / DB_PASSWORD provided by Railway PostgreSQL plugin
```

STEP 3 — RUN MIGRATIONS ON PRODUCTION

In Railway dashboard → service shell:

```
php artisan migrate --force  
php artisan storage:link
```

STEP 4 — DEPLOY FRONTENDS TO VERCEL

1. New Vercel project → GitHub repo → root: `frontend` → env: `VITE_API_URL=https://your-api.railway.app/api`
2. Second Vercel project → same repo → root: `website` → same env var

Backend (Laravel API) Complete

- Token authentication (login, logout, profile, photo upload, password change)
- Role-based access (admin, staff variants, member) with per-feature permission flags
- Full member management with documents, next of kin, status workflow
- Auto-creation of member portal accounts on approval (with duplicate email handling)
- Loan products (admin-only), full loan lifecycle, schedule generation, repayments
- Savings accounts, deposits/withdrawals, interest posting
- Withdrawal request workflow (pending → approved/rejected)
- Member shares and contributions with compliance tracking
- Welfare fund — contributions, claims, review, paid workflow
- Expenses, groups, guarantors, collateral, loan charges
- 6 financial reports, dividends, governance, news, inbox/chat
- Staff management with generated passwords and permission control
- Member portal endpoints: dashboard, loans (apply), savings, contributions, shares, withdrawal requests
- Public website endpoints: membership application, contact, news, chat

Admin Dashboard Complete

- Login with role-based redirect (members → /my, staff → /)
- Dashboard with live statistics and 6-month trend chart
- Full member lifecycle management — approve/reject with auto-generated credentials modal
- Savings compliance page with month picker and progress bars
- Complete loan lifecycle (all actions, statements, receipts)
- Savings detail with transactions; withdrawal request review
- Welfare fund management (contributions, claims, review, mark-paid)
- All financial reports, dividends, governance, groups
- News management (publish to public website)
- Inbox (contact messages + live chat)
- Staff & Access page (create staff, set permissions per feature)
- Settings — company, users, SMS logs (admin-only)
- Loan Products (admin-only)
- Sidebar: permission-filtered nav, admin badge, profile link with avatar
- Profile page (edit name/phone, upload photo, change password)

Member Portal Complete

- Overview dashboard with savings, loan eligibility, next payment, overdue alert
- My Loans — all loans with schedules; Apply for Loan modal (respects 3× savings limit)
- My Savings — accounts with transaction history; withdrawal request modal + request history
- Contributions history, shares summary
- Profile page (same as staff — edit info, photo, password)

Public Website Complete

- All 6 pages (Home, About, Products, Membership, News, Contact)
- Bush green + earthy amber design, fully responsive
- Membership form connected to API; news fetched dynamically from admin
- WhatsApp floating button, SVG logo, custom favicon

High Priority — Must Do Before Go-Live

Item	Description	Effort
Create Admin Account	Run <code>php artisan tinker</code> to create the first admin user, or build a seeder. No admin account means no one can log in to the dashboard.	15 min
Replace Website Placeholders	Phone numbers, email, social media links, and real photos need to be added to the website pages.	1–2 hours
Production Deployment	Deploy API to Railway and both frontends to Vercel. Set environment variables. Run migrations on production.	2–4 hours
Database Backups	Set up automated daily backups before real member data is entered. Railway has built-in backup options.	30 min
Custom Domain	Purchase and configure a domain (e.g. sycamorecoop.zm) for the website and admin dashboard.	1 hour
Storage on Production	Profile photos are stored on the server disk. For production, configure AWS S3 or Cloudflare R2 so uploads persist across deployments.	2–3 hours

Medium Priority — Next Phase

Item	Description	Effort
SMS Sending	Africa's Talking is configured but no sending logic is built. Need SMS for loan reminders, repayment alerts, contribution nudges.	1–2 days
PDF Export	Reports and statements display in browser but cannot be downloaded as PDF. Add dompdf (backend) or react-pdf (frontend).	1–2 days
Automated Penalty Application	Overdue loans are flagged but penalties are not auto-applied. A scheduled command (artisan schedule) could apply daily penalties to overdue loans.	1 day
Annual Savings Interest Bulk Post	Interest posting is manual per account. A bulk command to post interest to all savings accounts at year end would be useful.	1 day
Audit Trail UI	Activity logs stored in DB but no screen to view them.	1 day
WhatsApp Notifications	Planned but not built. Can use Twilio WhatsApp API or Africa's Talking.	2–3 days
Joining Fee	One-time fee on membership activation — not yet built. Currently deferred.	1 day

Future

Item	Description	Effort
Mobile Money Integration	MTN / Airtel Zambia for receiving contributions and repayments digitally.	2–3 weeks
Mobile App / PWA	Progressive web app for field officers to record repayments on the go.	3–4 weeks
Member Welfare Claims (Portal)	Members can currently submit welfare claims — this could be extended to allow them to track claim status in the portal.	1 day

Security — Before Going Live

The APP_KEY in the repo must NOT be used in production. Run `php artisan key:generate` on the production server. Set `APP_DEBUG=false` in production — never expose Laravel debug output publicly.

Database Driver Difference (Reports)

Reports use `DATE_FORMAT()` which is MySQL syntax. If production uses PostgreSQL (Railway), update DashboardController and ReportController to use `TO_CHAR()`. Test all reports after deployment.

File Storage on Deployment

Profile photos and documents are stored in `storage/app/public/`. On Railway, the filesystem is ephemeral — files are lost on redeploy. Before go-live, configure `FILESYSTEM_DISK=s3` and set up an S3-compatible bucket (AWS S3, Cloudflare R2, etc.).

CORS Configuration

CORS currently allows all origins (*). Before production, update `lms/config/cors.php` to list only the specific frontend domains.

Member Account Email Conflicts

If a member's email address is already used by a staff account, their member portal login email is automatically set to `[phone]@sycamore.member`. The approval modal shows the actual login email that was assigned.

Hardcoded company_id = 1

The public website endpoints hardcode `company_id = 1`. This is correct for Sycamore's single-tenant setup.

Contact for Technical Questions

All system code is in `c:\laragon\www\lms\`. For technical questions, contact the development team at **Stalwart Services Limited** (omrihabeenzu1@gmail.com).

